

Concours d'accès en première année du cycle d'ingénieurs pour les filières :

- Génie du Logiciel et des Systèmes Informatiques Distribués (GLSID)
- Ingénierie Informatique, Big Data et Cloud Computing (II-BDCC)

Session : Juillet 2019
Épreuve d'Informatique

Durée : 2 heures 30 min

Nom :	Prénom :
CIN :	N° d'examen :

Remarques importantes :

- Il est obligatoire de choisir votre ordre de préférence pour les deux filières **GLSID** et **II-BDCC** en précisant votre filière de premier choix et votre filière de deuxième choix dans le formulaire ci-dessous :

1^{er} choix :	2^{ème} choix :
Signature :	

- L'épreuve se compose de deux parties :
 - Partie QCM, (**Notée sur 50 points**)
 - Partie Algorithmique et Programmation, (**Notée sur 50 points**)
- L'usage de la calculatrice ou de tout autre appareil électronique est interdit.
- L'utilisation du blanco est strictement interdite.
- Aucun document n'est autorisé.
- Les extraits de code fournis dans l'épreuve sont écrits en langage C ou en langage Java.
- Aucune explication supplémentaire ne sera fournie aux candidats au cours de l'examen.
- Chaque question du QCM ne peut avoir qu'une seule réponse possible parmi les quatre choix. La réponse est à reporter dans la grille de réponse fournie (**page 14**) en cochant la case correspondante.
- Pour les exercices 1, 2 et 3 :
 - Les réponses aux questions doivent être rédigées dans des feuilles de réponses.
 - Les solutions algorithmiques peuvent être rédigées en utilisant un pseudo langage ou l'un des langages de programmation suivants : C, C++, C#, Java.
 - La clarté et la précision de votre solution algorithmique sera prise en considération.
- Sont à rendre la page de garde (**page 1**), la grille de réponses QCM (**page 14**) et les feuilles de rédaction.

QCM :

Pour cette partie, il ne peut y avoir qu'une seule réponse possible. La réponse est à reporter dans la grille de réponse fournie en page 14, en cochant la case correspondante :

1. Quelle est la formulation logique équivalente à la proposition mathématique suivante :
"Certains nombres réels sont des nombres rationnels" ?

A) $\exists x(\text{Reel}(x) \vee \text{Rationnel}(x))$ B) $\forall x(\text{Reel}(x) \rightarrow \text{Rationnel}(x))$
C) $\exists x(\text{Reel}(x) \wedge \text{Rationnel}(x))$ D) $\exists x(\text{Rationnel}(x) \rightarrow \text{Reel}(x))$

2. Quelle est la formulation logique qui correspond à la proposition suivante :
"Aucun de mes amis n'est parfait" ?

On adopte les notations suivantes : x désigne une personne, $F(x)$ représente l'assertion « x est mon ami » et $P(x)$ représente l'assertion « x est parfait ».

A) $\exists x(F(x) \wedge \neg P(x))$ B) $\exists x(\neg F(x) \wedge P(x))$
C) $\exists x(\neg F(x) \wedge \neg P(x))$ D) $\neg \exists x(F(x) \wedge P(x))$

3. Soit le prédicat $R(x, y, t)$ qui représente l'affirmation « une personne x peut rencontrer une personne y au moment t ». Lequel des énoncés ci-dessous exprime le mieux le sens de la formule $\forall x \exists y \exists t(\neg R(x, y, t))$?

A) Tout le monde peut rencontrer une personne à un moment donné.
B) Aucun ne peut rencontrer tout le monde tout le temps.
C) Tout le monde ne peut pas rencontrer une personne tout le temps.
D) Aucun ne peut rencontrer une personne à un moment donné.

4. Parmi les propositions suivantes, laquelle est la formule logique la plus appropriée pour représenter la déclaration : "Les ornements en or et en argent sont précieux" ? Les notations suivantes sont utilisées : $O(x)$: x est un ornement en Or, $A(x)$: x est un ornement en Argent et $P(x)$: x est Précieux.

A) $\forall x(P(x) \rightarrow (O(x) \wedge A(x)))$ B) $\forall x((O(x) \wedge A(x)) \rightarrow P(x))$
C) $\exists x((O(x) \wedge A(x)) \rightarrow P(x))$ D) $\forall x((O(x) \vee A(x)) \rightarrow P(x))$

5. Soit $G(x)$ un prédicat qui indique que x est un graphe. Soit $C(x)$ un prédicat qui indique que x est connecté. Laquelle des expressions logiques ne représente pas l'énoncé suivant : **"Tous les graphes ne sont pas connectés"**?

- A) $\neg \forall x (G(x) \rightarrow C(x))$
- B) $\exists x (G(x) \wedge \neg C(x))$
- C) $\forall x (G(x) \rightarrow \neg C(x))$
- D) *Aucune des expressions ci-dessus.*

6. Soit l'extrait de code suivant :

```
short a = -2;
unsigned short b = -a;
int x = -2;
unsigned y = -x;
if(a<(unsigned short)b)
    printf("a < b\t") ;
else
    printf("a >= b\t") ;
if((unsigned)x<y)
    printf("et x < y\n") ;
else
    printf("et x >= y\n") ;
```

L'exécution de ce code produit l'affichage :

- A) $a < b$ et $x < y$
- B) $a < b$ et $x \geq y$
- C) $a \geq b$ et $x < y$
- D) $a \geq b$ et $x \geq y$

7. Soit l'extrait de code suivant :

```
unsigned char x = 0,i=x;
int j=0;
for (; i >= 0; i--) if (j++ == 5)break;
printf("i = %hu, j = %d\n",i,j);
```

L'exécution de ce code produit l'affichage :

- A) $i = -1, j = 1$
- B) $i = -1, j = 5$
- C) $i = 251, j = 6$
- D) $i = 252, j = 5$

8. Soit l'extrait de code suivant :

```
int a=256;
char *x= (char *)&a;
*x++ = 1;
*x =x[0]++;
printf("a=%d\n", a);
```

L'exécution de ce code produit l'affichage :

- | | |
|----------|--------------------------|
| A) a=257 | B) a=256 |
| C) a=513 | D) Erreur de compilation |

9. Soit le code suivant :

```
int f1(int *p,int **pp){
    int y, z=++**pp;
    y = ++*p;
    return ++y + z;
}
int main(){
    int c=4, *b=&c, **a=&b;
    printf("%d\n", c+f1(b, a));
    return 0;
}
```

L'exécution de ce code produit l'affichage :

- | | |
|-------|-------|
| A) 16 | B) 17 |
| C) 18 | D) 19 |

10. Soit le code suivant :

```
int main(){
    int a[][3] = {1, 2, 3, 4, 5, 6}, (*ptr)[3] = a;
    printf("%d,%d", (*ptr)[1], (*ptr)[2]);
    ptr++;
    printf(",%d,%d\n", (*ptr)[1], (*ptr)[2]);
    return 0 ;
}
```

L'exécution de ce code produit l'affichage :

- | | |
|---------------|-----------------------|
| A) 2, 3, 2, 3 | B) 2, 3, 5, 6 |
| C) 2, 3, 3, 4 | D) Erreur d'exécution |

11. Soit le code suivant :

```
int main()
{
    int i, *p;
    p = (int *) malloc(4 * sizeof(int));
    for (i=0; i<4; *(p+i)=i, i++);
    printf("(%d", *p++);
    printf(", %d", (*p)++);
    printf(", %d", *p);
    printf(", %d", *++p);
    printf(", %d)\n", ++*p);
    return 0;
}
```

L'exécution de ce code produit l'affichage :

- A) Erreur de compilation B) (0, 1, 2, 3, 4)
- C) Erreur d'exécution D) (0, 1, 2, 2, 3)
12. Lequel des algorithmes de tri suivants n'est naturellement pas un algorithme « diviser pour régner »?
- A) Tri fusion B) Tri rapide
- C) Tri par Tas D) Aucn des trois algorithmes
13. Soit A un arbre binaire de recherche d'entiers. Quel est le parcours qui fait ressortir les entiers de A dans un ordre croissant ?
- A) Parcours en ordre préfixe B) Parcours en ordre postfixe
- C) Parcours en ordre infixe D) Parcours en largeur
14. Pour gérer les appels à des fonctions récursives, la structure de données utilisée est :
- A) File B) Pile
- C) File de priorité D) Tableau

15. En l'absence de condition de sortie dans une fonction récursive, l'appel à une telle fonction entraîne :

- A) Erreur de compilation
- B) Erreur logique
- C) Erreur d'exécution
- D) Pas d'erreur

16. Pour deux solutions algorithmiques récursive et itérative équivalentes, laquelle des affirmations suivantes est vraie?

- A) La récursion est toujours meilleure que l'itération.
- B) La récursion utilise plus de mémoire que l'itération.
- C) La récursion utilise moins de mémoire que l'itération.
- D) L'itération est toujours meilleure et plus simple que la récursion.

17. On considère la fonction suivante :

```
void f2(int a)
{
    if(a>0) f2(a-1);
    printf("%d ",a);
}
```

Quel est le résultat qui sera affiché suite à l'appel f2(5):

- A) 5 4 3 2 1
- B) 0 1 2 3 4 5
- C) 4
- D) 5 4 3 2 1 0

18. On considère la fonction suivante :

```
void f3(int a)
{
    printf("%d ",a);
    if(a>0) f3(a-1);
    printf("%d ",a);
}
```

Quel est le résultat qui sera affiché suite à l'appel f3(5):

- A) 5 4 3 2 1 0 1 2 3 4 5
- B) 5 4 3 2 1 0 0 1 2 3 4 5
- C) 5 4 3 2 1 0 5
- D) 5 4

19. On considère le programme suivant :

```
int min(int x, int y){ return x<y?x :y ; }
int f4(char *s1, int n1, char *s2, int n2){
    int T1[26] = {0} , T2[26] = {0},i, count = 0;
    for (i = 0; i < n1; i++)T1[s1[i] - 'a']++;
    for (i = 0; i < n2; i++)T2[s2[i] - 'a']++;
    for (i = 0; i < 26; i++)count += (min(T1[i], T2[i]));
    return count;
}
int main(){
    char *s1 = "glsid"; char *s2 = "iibdcc";
    int n1 = strlen(s1), n2 = strlen(s2);
    printf("%d",f4(s1, n1, s2, n2));
    return 0;
}
```

Quel est le résultat affiché par ce programme:

- A) 3 B) 4
C) 1 D) 2

20. On considère le programme suivant :

```
void f5(int T[],int n){
    int i=0,j=n,temp;
    while(1){
        if(T[i]>=T[j-1]) {
            j--;
            temp=T[i];  T[i]=T[j-1];  T[j-1]=temp;
        }
        else i++;
        if(i>j) break;
    }
}

int main(){
    int T[]={3,7,9,6,8,11,33,5,20,2},i;
    f5(T,10);
    for(i=0;i<10;i++) printf("%d  ",T[i]);
    return 0;
}
```

Quel est le résultat affiché par ce programme:

- A) 5 7 9 6 8 11 33 20 3 2 B) 5 7 9 6 8 33 11 20 3 2
- C) 2 3 5 6 7 8 9 11 33 20 D) 2 3 5 6 7 8 9 11 20 33

21. Soit **fil** une variable structurée de type **File** qui représente la structure de données file d'attente. la variable **fil** est composée des champs **First** et **Last** qui représentent respectivement l'adresse du premier élément et l'adresse du dernier élément de la file. L'initialisation des champs **First** et **Last** à **NULL** d'une file juste après sa création, la met à l'état vide. On dispose aussi des deux fonctions suivantes pour manipuler une file d'attente :

- La fonction **enqueue** permet d'ajouter un élément à la fin de la file d'attente
- La fonction **dequeue** permet d'enlever le premier élément de la file d'attente

On considère la fonction suivante :

```
void f6(int n)
{
    int i ,a,b;
    File fil ;
    fil.Last=fil.First=NULL ;
    enqueue(&fil,0);
    enqueue(&fil,1);
    for (i = 0; i < n; i++){
        a=dequeue(&fil);
        b=dequeue(&fil);
        enqueue(&fil,b);
        enqueue(&fil,a + b);
        printf("%d ",a);
    }
}
```

Quel est le résultat affiché suite à l'appel f6(6):

- | | |
|----------------------|----------------|
| A) 0 1 1 2 2 3 3 5 5 | B) 0 1 2 3 5 |
| C) 0 1 1 2 3 5 | D) 0 1 2 3 4 5 |

22. On considère la fonction suivante :

```
int f7(int n){
    if(n<=2) return 1;
    return f7(n-1)+f7(n-2);
}
```

Quel est la valeur retournée suite à l'appel f7(6):

- | | |
|-------|-------|
| A) 12 | B) 11 |
| C) 9 | D) 8 |

23. On considère le programme suivant :

```
void f8(int a,int *b)
{
    a++;
    *b=a++;
    a=*b;
}

int main(){
    int a=5, b=5;
    f8(a,&b);
    printf("%d %d",a,b);
    return 0;
}
```

Quel est le résultat affiché par le programme:

- | | |
|--------|--------|
| A) 5 5 | B) 7 7 |
| C) 5 6 | D) 5 7 |

24. On considère la fonction suivante :

```
int f9(int T[],int n)
{
    int *a=T, i;
    for(i=0;i<n;i++){
        *a=*a+T[i];
        a=a+1;
    }
}

int main()
{
    int T[]={1,2,3,4,5};
    int i;
    f9(T,5);
    for(i=0;i<5;i++)
        printf("%d ",T[i]);
    return 0;
}
```

Quel est le résultat affiché dans le main :

- | | |
|---------------------------|--------------------------|
| A) 2 4 6 8 10 | B) 1 2 3 4 5 |
| C) Débordement de mémoire | D) Erreur de compilation |

25. On considère le programme suivant :

```
int f10(int T[], int n, int k) {
    int a = 0, i = 0, j = 1;
    while (i < n && j < n) {
        if(j<=i) j=i+1;
        while (j < n && (T[j] - T[i]) < k)
            j++;
        a += (n - j); i++;
    }
    return a;
}

int main() {
    int T[]={5,6,7,8} ;
    int n=4, k=2, a;
    a=f10(T,n,k);
    printf("%d",a);
    return 0;
}
```

Quel est le résultat affiché par ce programme :

- | | |
|------|------|
| A) 3 | B) 2 |
| C) 5 | D) 6 |

Exercice 1 (15 points) :

Soit un tableau T contenant des entiers positifs et négatifs.

Ecrire un programme qui permet d'extraire de T le sous-tableau U dont la somme est la plus proche à 0. Il peut y avoir plusieurs sous-tableaux de ce type, il suffit dans ce cas d'en générer un seul.

Exemples :

- Entrée : $T = \{-1, 3, 4, -7, 12\}$, Sortie $U = \{3, 4, -7\}$
- Entrée : $T = \{-1, 1, 4, -3, 3, 7\}$, Sortie $U = \{-1, 1\}$ ou bien $U = \{-3, 3\}$
- Entrée : $T = \{-2, 5, 7, 9, 1\}$, Sortie $U = \{1\}$

Exercice 2 (17 points) :

On souhaite écrire un algorithme qui permet de minimiser une fonction mathématique quelconque à plusieurs variables en utilisant l'algorithme de descente du gradient. Dans notre problème nous considérons l'exemple de la fonction : $f(x, y) = 3x^3 + 7y^2 - 8$

L'algorithme de descente du gradient, très utilisé dans plusieurs problèmes d'optimisation, particulièrement en machines Learning, est un algorithme itératif qui consiste à chercher les valeurs de x et y qui minimisent la fonction f . Pour cela, pour chaque itération, il faut calculer les gradients Dx et Dy de la fonction f , respectivement dans les directions x et y . Ensuite on avance x et y dans le sens opposé de leurs gradients respectifs pondérés par un coefficient compris entre 0 et 1. Ce coefficient représente la vitesse d'apprentissage (**Learning Rate**). L'algorithme fonctionne comme suit :

Etape 1. Partir des valeurs initiales quelconques de x et de y

Etape 2. Calculer les gradients représentés par les dérivées partielles Dx et Dy de la fonction f , respectivement par rapport à x et y .

Etape 3. On avance x et y dans le sens opposé de leurs gradients respectifs Dx et Dy en multipliant ces gradients par un coefficient compris entre 0 et 1 appelé vitesse d'apprentissage pour éviter l'oscillation autour de la solution. Dans notre cas, nous utiliserons une vitesse d'apprentissage égale à 0.1.

Etape 4. Répéter les étapes 2 et 3 jusqu'à ce que l'on trouve les valeurs de x et y qui minimisent la fonction. Chaque itération correspond à une période d'optimisation appelée « Epoch ».

Questions :

1. Ecrire les deux fonctions qui permettent de retourner les dérivées partielles de la fonction f par rapport à x et y .
2. Ecrire le code de l'algorithme de la descente du gradient qui permet de minimiser la fonction f .
3. Pour des fonctions plus complexes présentant des minimums locaux en plus du minimum global, l'algorithme de descente du gradient présenté précédemment risque de s'arrêter dans un minimum local au lieu de continuer à chercher le vrai minimum global. Quelle est la solution que vous proposez pour éviter ce problème ?

Exercice 3 (18 points) :

On souhaite mettre en place une application qui simule la gestion de la liste des applications ouvertes sous un système d'exploitation (Windows, Linux, ...). Plus précisément, on souhaite gérer le comportement obtenu par l'appui sur la combinaison de touches **ALT + TAB** qui permet de changer l'application courante (application active) et l'ordre des applications les plus récemment utilisées. Pour se faire, on propose le modèle suivant :

Soit un entier **k** et un tableau **listeApps[]** de **N** entiers contenant les **identifiants** des **N** applications ouvertes dans le système où :

- **listeApps[0]** est l'application en cours d'utilisation (l'application active)
- **listeApps[1]** est l'application la plus récemment utilisée
- **listeApps[N - 1]** est l'application la moins récemment utilisée.

La tâche principale à programmer consiste à mettre à jour le contenu du tableau **listeApps** puis à imprimer son contenu lorsque l'utilisateur appuie sur la touche **Alt + la touche Tab** répétée exactement **k fois** (**k > 0**). Ainsi, après avoir appuyé sur les touches **Alt + (k fois) Tab**, le pointeur d'ouverture de l'application se déplace depuis l'index 0 vers la droite, en fonction du nombre d'appuis **k**. L'application sur laquelle l'appui se termine, se déplace vers l'index 0 pour devenir ainsi l'application active. Toutes les applications situées avant l'index **k** devront être décalées pour qu'elles soient moins récemment utilisées que la nouvelle application active dont l'identifiant prendra l'index 0.

Exemples :

*Entrée : listeApps [] = {3, 5, 2, 4, 1, 7, 6}, K = 3 (ALT + 3*TAB)*

Sortie : 4, 3, 5, 2, 1, 7, 6

*Entrée : listeApps [] = {5, 7, 2, 3, 4, 1, 6, 8}, K = 10 (ALT + 10*TAB)*

Sortie : 2, 5, 7, 3, 4, 1, 6, 8

Questions :

1. Ecrire une fonction nommée **imprimerListeApps** qui affiche la liste des identifiants des applications ouvertes dans l'ordre de la plus récemment jusqu'au moins récemment utilisé.
2. Ecrire une fonction nommée **modifierAppCourante** qui étant donnée le nombre k et la liste des ids des applications ouvertes, modifie l'application courante, change l'ordre des autres applications et affiche la liste des ids dans l'ordre de la plus récente à la moins récente.
3. On souhaite pour chaque application ouverte, compter le nombre de fois qu'elle a été rendue active et cumuler le temps total pendant lequel elle l'a été.
 - a. Proposer sous la forme d'une représentation graphique, une structure de données qui permet de faire ces calculs.
 - b. Donner les déclarations nécessaires pour définir cette structure de données,
 - c. Récrire les fonctions **imprimerListeApps** et **modifierAppCourante** en tenant compte des calculs demandés.
4. On souhaite, en plus, prendre en charge les possibilités d'ouverture et de fermeture de nouvelles applications.
 - a. Quels sont les changements à recommander au niveau de la structure de données pour tenir compte de ces possibilités ? Les changements proposés doivent permettre l'optimisation de la mémoire et minimiser les opérations de décalage. Illustrer vos choix par une représentation graphique.
 - b. Ecrire une fonction nommée **fermerAppCourante** qui met à jour et imprime la liste des applications ouvertes suite à la fermeture de l'application courante,
 - c. Ecrire une fonction nommée **ouvrirApplication** qui étant donnée l'id de l'application à ouvrir, met à jour et imprime la liste des applications ouvertes.

Concours d'accès en 1^{ère} année des Cycles d'Ingénieurs GLSID et II-BDCC

Session Juillet 2019

Epreuve d'informatique

Numéro de l'examen :

Nom :

Prénom :

Signature :

Epreuve d'informatique

Grille de réponses pour la partie QCM

- **Cocher, avec stylo à encre, la case correspondante à la bonne réponse.**
- **Réponse juste : 2 pts**
- **Réponse fausse : 0 pt**

Note :

	A	B	C	D	E
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					

	A	B	C	D	E
14					
15					
16					
17					
18					
19					
20					
21					
22					
23					
24					
25					